

Name: Abhinav Garg

ID: 2023ucp1706

PDC Assignment-2

Ans.1.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#define SIZE 1000
#define MIN_VAL 1
#define MAX_VAL 100

int main() {
    double *a, *b;
    double dot_serial = 0.0;
    double dot_parallel = 0.0;
    double start, end;
    a = (double *)malloc(SIZE * sizeof(double));
    b = (double *)malloc(SIZE * sizeof(double));
    srand(42);
    for (int i = 0; i < SIZE; i++) {
        a[i] = MIN_VAL + rand() % MAX_VAL;
        b[i] = MIN_VAL + rand() % MAX_VAL;
    }
    start = omp_get_wtime();
    for (int i = 0; i < SIZE; i++) {
        dot_serial += a[i] * b[i];
    }
    end = omp_get_wtime();
    double time_serial = end - start;
    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:dot_parallel)
    for (int i = 0; i < SIZE; i++) {
        dot_parallel += a[i] * b[i];
    }
    end = omp_get_wtime();
    double time_parallel = end - start;
    printf("Serial dot product = %.2f\n", dot_serial);
    printf("Parallel dot product = %.2f\n", dot_parallel);
    printf("Serial time = %f s\n", time_serial);
    printf("Parallel time = %f s\n", time_parallel);
    free(a);
    free(b);

    return 0;
}
```

Output:

```
mnit@mnit-OptiPlex-5040:~/pdc_assignment_2$ export OMP_NUM_THREADS=4
mnit@mnit-OptiPlex-5040:~/pdc_assignment_2$ gcc -o ans1 -fopenmp ans1.c
mnit@mnit-OptiPlex-5040:~/pdc_assignment_2$ ./ans1
Dot product (serial)    = 2509012.00
Dot product (parallel) = 2509012.00
Time (serial)          = 0.000011 seconds
Time (parallel)        = 0.000463 seconds
```

Ans.2.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int nthreads;
    #pragma omp parallel
    {
        #pragma omp single
        nthreads = omp_get_num_threads();
    }
    int random_numbers[nthreads];
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        unsigned int seed = 1234 + tid;
        random_numbers[tid] = (rand_r(&seed) % 100) + 1;
        #pragma omp barrier

        #pragma omp single
        {
            int max = random_numbers[0];
            for (int i = 1; i < nthreads; i++) {
                if (random_numbers[i] > max)
                    max = random_numbers[i];
            }
            printf("Generated random numbers:\n");
            for (int i = 0; i < nthreads; i++) {
                printf("Thread %d: %d\n", i, random_numbers[i]);
            }

            printf("Maximum value = %d\n", max);
        }
    }
    return 0;
}
```

Output:

```
mnit@mnit-OptiPlex-5040:~/pdc_assignment_2$ ./ans2
Generated random numbers:
Thread 0: 8
Thread 1: 38
Thread 2: 44
Thread 3: 49
Thread 4: 79
Thread 5: 33
Maximum value = 79
```

Ans.3.

Code:

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int counter = 0;
        for (int i = 0; i < 10; i++) {
            counter += tid;
        }
        #pragma omp critical
        {
            printf("Thread %d: Counter = %d\n", tid, counter);
        }
    }
    return 0;
}
```

Output:

```
mnit@mnit-OptiPlex-5040:~/pdc_assignment_2$ gcc -o ans3 -fopenmp ans3.c
[1]+  Done                  gedit ans3.c
mnit@mnit-OptiPlex-5040:~/pdc_assignment_2$ ./ans3
Thread 0: Counter = 0
Thread 2: Counter = 20
Thread 3: Counter = 30
Thread 6: Counter = 60
Thread 5: Counter = 50
Thread 1: Counter = 10
Thread 4: Counter = 40
Thread 7: Counter = 70
```